ICT2223 – Internet of Things

Practical 03: ARDUINO LEDs AND 7-SEGMENT DISPLAYS

Prepared By – Ms.Sandaruwani Pathirage

Learning Outcomes

By the end of this practical, you should be able to:

- Identify and use Arduino digital output pins.
- Connect LEDs to an Arduino UNO using a breadboard.
- Control multiple LEDs using Arduino code.
- Identify the segments of a 7-segment display.
- Interface a 7-segment display with Arduino.
- Display numbers from **0–9**.
- Install and use the **SevSeg** library.
- Interface a 4-digit 7-segment display.
- Display **1234** and other number patterns.

ACTIVITY 1 — TRAFFIC LIGHT USING ARDUINO

Build a simple traffic-light system using three LEDs and control them using an Arduino UNO.

Components Required

- Arduino UNO
- Breadboard
- Red LED
- Yellow LED

- Green LED
- Jumper wires
- Arduino IDE

Pin Connections

LED	Arduino Pin
Red	Digital Pin 9
Yellow	Digital Pin 8
Green	Digital Pin 7
GND	Arduino GND

Step 1 – Prepare the Circuit

Place the three LEDs on the breadboard.

Step 2 – Connect Ground

Connect the ground side of the LEDs to the Arduino **GND**.

Step 3 – Connect the LEDs

Connect:

- Red LED → Pin 9
- Yellow LED → Pin 8
- Green LED → Pin 7

Step 4 – Program the Arduino

Open Arduino IDE and create a new sketch.

Configure pins **7, 8 and 9** as digital outputs.

Source Code

```
// Traffic Light using Arduino UNO

int redLED = 9;
int yellowLED = 8;
int greenLED = 7;

void setup()
{
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
}

void loop()
{
  // RED
  digitalWrite(redLED, HIGH);
  digitalWrite(yellowLED, LOW);
  digitalWrite(greenLED, LOW);
  delay(5000);

  // YELLOW
  digitalWrite(redLED, LOW);
  digitalWrite(yellowLED, HIGH);
```

```
  digitalWrite(greenLED, LOW);
  delay(2000);

  // GREEN
  digitalWrite(redLED, LOW);
  digitalWrite(yellowLED, LOW);
  digitalWrite(greenLED, HIGH);
  delay(5000);

  // YELLOW
  digitalWrite(redLED, LOW);
  digitalWrite(yellowLED, HIGH);
  digitalWrite(greenLED, LOW);
  delay(2000);
}
```

Step 5 – Upload and Test

Upload the program to the Arduino and observe the LEDs.

Task

Modify the program so that the LEDs operate as a traffic-light sequence:

Red → Yellow → Green → Yellow → Red

Choose suitable delay times between each state.

Expected Output

The LEDs should turn ON/OFF according to the programmed traffic-light sequence.

ACTIVITY 2 — SINGLE 7-SEGMENT DISPLAY

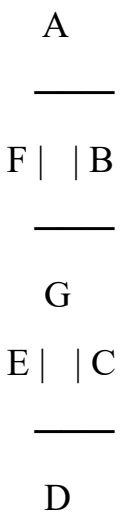
Interface a single 7-segment display with Arduino UNO and display numbers from **0 to 9**.

Components Required

- Arduino UNO
- Single 7-segment display
- Breadboard
- Jumper wires
- Resistor
- Arduino IDE

2.1 Understand the 7-Segment Display

A 7-segment display contains seven main segments:



The display also contains a **decimal point (DP)**.

The segments are:

- A
- B
- C
- D
- E
- F
- G
- DP

2.2 Arduino Pin Connections

Use the following connections

Arduino UNO	**7-Segment**
Pin 7	A
Pin 6	B
Pin 3	C
Pin 4	D
Pin 5	E
Pin 8	F
Pin 9	G
Pin 2	DP
GND	Common terminal(s)

2.3 Programming Task

Start with below Arduino program.

Source Code

```
// Single 7-Segment Display
// Display numbers 0-9

int A = 7;
int B = 6;
int C = 3;
int D = 4;
int E = 5;
int F = 8;
int G = 9;
int DP = 2;

void setup()
{
  pinMode(A, OUTPUT);
  pinMode(B, OUTPUT);
  pinMode(C, OUTPUT);
  pinMode(D, OUTPUT);
```

```
  pinMode(E, OUTPUT);
  pinMode(F, OUTPUT);
  pinMode(G, OUTPUT);
  pinMode(DP, OUTPUT);
}

void loop()
{
  // 0
  digitalWrite(A, HIGH);
  digitalWrite(B, HIGH);
  digitalWrite(C, HIGH);
  digitalWrite(D, HIGH);
  digitalWrite(E, HIGH);
  digitalWrite(F, HIGH);
  digitalWrite(G, LOW);
  delay(1000);

  // 1
  digitalWrite(A, LOW);
  digitalWrite(B, HIGH);
  digitalWrite(C, HIGH);
  digitalWrite(D, LOW);
  digitalWrite(E, LOW);
  digitalWrite(F, LOW);
```

```
digitalWrite(G, LOW);
delay(1000);

// 2
digitalWrite(A, HIGH);
digitalWrite(B, HIGH);
digitalWrite(C, LOW);
digitalWrite(D, HIGH);
digitalWrite(E, HIGH);
digitalWrite(F, LOW);
digitalWrite(G, HIGH);
delay(1000);

// 3
digitalWrite(A, HIGH);
digitalWrite(B, HIGH);
digitalWrite(C, HIGH);
digitalWrite(D, HIGH);
digitalWrite(E, LOW);
digitalWrite(F, LOW);
digitalWrite(G, HIGH);
delay(1000);

// 4
digitalWrite(A, LOW);
```

```
digitalWrite(B, HIGH);
digitalWrite(C, HIGH);
digitalWrite(D, LOW);
digitalWrite(E, LOW);
digitalWrite(F, HIGH);
digitalWrite(G, HIGH);
delay(1000);

// 5
digitalWrite(A, HIGH);
digitalWrite(B, LOW);
digitalWrite(C, HIGH);
digitalWrite(D, HIGH);
digitalWrite(E, LOW);
digitalWrite(F, HIGH);
digitalWrite(G, HIGH);
delay(1000);

// 6
digitalWrite(A, HIGH);
digitalWrite(B, LOW);
digitalWrite(C, HIGH);
digitalWrite(D, HIGH);
digitalWrite(E, HIGH);
digitalWrite(F, HIGH);
```

```
digitalWrite(G, HIGH);
delay(1000);

// 7
digitalWrite(A, HIGH);
digitalWrite(B, HIGH);
digitalWrite(C, HIGH);
digitalWrite(D, LOW);
digitalWrite(E, LOW);
digitalWrite(F, LOW);
digitalWrite(G, LOW);
delay(1000);

// 8
digitalWrite(A, HIGH);
digitalWrite(B, HIGH);
digitalWrite(C, HIGH);
digitalWrite(D, HIGH);
digitalWrite(E, HIGH);
digitalWrite(F, HIGH);
digitalWrite(G, HIGH);
delay(1000);

// 9
digitalWrite(A, HIGH);
```

```
  digitalWrite(B, HIGH);
  digitalWrite(C, HIGH);
  digitalWrite(D, HIGH);
  digitalWrite(E, LOW);
  digitalWrite(F, HIGH);
  digitalWrite(G, HIGH);
  delay(1000);
}
```

Task 1

Modify the program to display:

0

Task 2

Modify the program to display:

1, 2, 3, 4, 5

Task 3

Continue the program to display:

6, 7, 8, 9

2.4 Using the SevSeg Library

Install the **SevSeg** Arduino library.

Use SevSeg.h to simplify the display of numbers from 0–9.

Task 4

Using the **SevSeg** library:

1. Configure the display.
2. Display numbers from **0 to 9**.
3. Compare this approach with your previous program.

Which approach requires less code?

- Controlling each segment manually
- Using the SevSeg library

```
#include <SevSeg.h>

SevSeg sevseg;

void setup()
{
  byte numDigits = 1;

  byte digitPins[] = {};

  byte segmentPins[] = {
    7, 6, 3, 4, 5, 8, 9, 2
  };
```

```
  bool resistorsOnSegments = true;
  byte hardwareConfig = COMMON_CATHODE;

  sevseg.begin(
    hardwareConfig,
    numDigits,
    digitPins,
    segmentPins,
    resistorsOnSegments
  );

  sevseg.setBrightness(90);
}

void loop()
{
  for (int number = 0; number <= 9; number++)
  {
    sevseg.setNumber(number);

    unsigned long startTime = millis();

    while (millis() - startTime < 1000)
    {
```

```
    sevseg.refreshDisplay();
  }
 }
}
```

ACTIVITY 3 — 4-DIGIT 7-SEGMENT DISPLAY

Interface a 4-digit 7-segment display with Arduino UNO and display multi-digit numbers.

Components Required

- Arduino UNO
- 4-digit 7-segment display
- Breadboard
- Jumper wires
- 4 × 200 Ω resistors
- Arduino IDE
- SevSeg library

3.1 Understand the Display

A typical 4-digit 7-segment display contains **12 pins**.

Four pins control the individual digits:

- D1
- D2
- D3

- D4

The remaining pins control the individual segments.

This arrangement is illustrated in the pin-out diagram below.



3.2 Circuit Setup

1. Place the 4-digit display on the breadboard.
2. Identify the 12 pins.
3. Identify **D1, D2, D3 and D4**.
4. Identify the segment connections.
5. Connect the display to the Arduino UNO.
6. Use the four resistors as shown in the provided circuit.
7. Check all connections before powering the circuit.

3.3 Install the SevSeg Library

Install the **SevSeg** library in Arduino IDE.

3.4 Programming Tasks

Task 1 – Display 0–9

Write an Arduino program to display:

0 → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9

```
#include <SevSeg.h>

SevSeg sevseg;

void setup()
{
  byte numDigits = 4;

  byte digitPins[] = {
    10, 11, 12, 13
  };

  byte segmentPins[] = {
    2, 3, 4, 5, 6, 7, 8, 9
```

```
  };

  bool resistorsOnSegments = true;

  byte hardwareConfig = COMMON_CATHODE;

  sevseg.begin(
    hardwareConfig,
    numDigits,
    digitPins,
    segmentPins,
    resistorsOnSegments
  );

  sevseg.setBrightness(90);
}

void loop()
{
  for (int number = 0; number <= 9; number++)
  {
    sevseg.setNumber(number);

    unsigned long startTime = millis();
```

```
    while (millis() - startTime < 1000)
    {
      sevseg.refreshDisplay();
    }
  }
}
```

Task 2 – Display 1234

Write a program to display:

1234

```
#include <SevSeg.h>

SevSeg sevseg;

void setup()
{
  byte numDigits = 4;

  byte digitPins[] = {
    10, 11, 12, 13
  };

  byte segmentPins[] = {
    2, 3, 4, 5, 6, 7, 8, 9
  };
```

```
  bool resistorsOnSegments = true;

  byte hardwareConfig = COMMON_CATHODE;

  sevseg.begin(
    hardwareConfig,
    numDigits,
    digitPins,
    segmentPins,
    resistorsOnSegments
  );

  sevseg.setBrightness(90);
}

void loop()
{
  sevseg.setNumber(1234);

  sevseg.refreshDisplay();
}
```

Task 3 – Display Number Patterns

Create your own number patterns.

Examples:

0000

1111

2222

1234

4321

2025

You may create your own pattern as an additional challenge.

```
#include <SevSeg.h>

SevSeg sevseg;

void setup()
{
  byte numDigits = 4;

  byte digitPins[] = {
    10, 11, 12, 13
  };

  byte segmentPins[] = {
    2, 3, 4, 5, 6, 7, 8, 9
  };

  bool resistorsOnSegments = true;

  byte hardwareConfig = COMMON_CATHODE;
```

```
  sevseg.begin(
    hardwareConfig,
    numDigits,
    digitPins,
    segmentPins,
    resistorsOnSegments
  );

  sevseg.setBrightness(90);
}

void loop()
{
  sevseg.setNumber(1234);

  sevseg.refreshDisplay();
}
```

4. Practical Challenge

Complete the following challenge independently.

Challenge: Digital Counter

Modify the 4-digit display program to create a simple counter.

The display should show:

0000

0001

0002

0003

...

Continue until:

9999

Extension

Change the delay so that the counting speed can be controlled.